

Event Tracking Plan & Event Schema

Onboarding Flow Optimization A/B Test

Aishwarya Mishra

August 2026

1. Business Understanding
 - 1.1 The Business
 - 1.2 The Problem
 - 1.3 The Opportunity (Illustrative)
 - 1.4 Hypothesis
 - 1.5 Experiment Design
 - 1.6 How Steps Are Cut
2. KPIs
 - 2.1 Primary Metric
 - 2.2 Supporting Metrics
 - 2.3 Guardrail Metrics (must not regress)
3. Data Platform Requirement
 1. Design Principles
 2. Envelope (present on every event)
 3. Identity & Randomization
 4. Event Catalog
 - 4.1 Funnel & Onboarding Events
 - 4.2 Product Engagement Events
 - 4.3 Monetization Events
 - 4.4 Quality & Guardrail Events
5. Step Maps by Variant
6. Metric → Event Lineage

1. Business Understanding

1.1 The Business

A B2C mobile app that teaches English using AI. New users go through an onboarding flow (account creation, level assessment, goal setting, personalization questions, notification permission, etc.) before they reach the actual product.

1.2 The Problem

The current onboarding flow has **13 sequential steps**. Only **60% of users who start onboarding finish it**, against an industry benchmark of **~80%**. Every extra step is a point where a new user can quit before ever experiencing the product — and since most of these users were already paid for via acquisition spend (ads, referrals), every drop-off is wasted spend.

Critically, the **post-onboarding paid conversion rate (6%) is already healthy and in line with industry**. The bottleneck is not the product or the price — it is the number of people who never make it far enough to be counted.

1.3 The Opportunity (Illustrative)

Using illustrative figures of 100,000 new users/month and \$30 average monthly revenue per paying user:

Scenario	Completion Rate	Users who Finish	Paying Users (6%)	Monthly Revenue
Current (13 steps)	60%	60,000	3,600	\$108,000
Target (industry)	80%	80,000	4,800	\$144,000

Closing the gap is worth an estimated **+\$36,000/month (+\$432,000/year)** — from the *same* user base, with zero additional acquisition spend.

1.4 Hypothesis

Reducing the number of onboarding steps from 13 will reduce drop-off, increasing onboarding completion, activation, and downstream paid conversion. The design also isolates *step count* from *step content* — steps are trimmed by removing or deferring genuinely low-value steps (see 1.6), not just truncating the flow, so a win can be attributed to friction reduction rather than to accidentally dropping a specific confusing step.

1.5 Experiment Design

	A — Control	B — Medium	C — Short
Steps	13 (unchanged)	7	5
Traffic split	~33%	~33%	~33%
Platform	iOS + Android, balanced	iOS + Android, balanced	iOS + Android, balanced
Randomization unit	Per new user, at first app open		

1.6 How Steps Are Cut

Steps are bucketed as **Must-have** (account creation, level check — needed before personalization is possible), **Can-wait** (notification permission, detailed goals — deferred to after first value), and **Nice-to-have / combinable** (merged into other screens or dropped). B keeps must-have + some can-wait steps; C keeps only must-have steps.

2. KPIs

2.1 Primary Metric

- **Onboarding completion rate** — % of users who start onboarding and finish it. This is the headline number: it directly measures whether shorter onboarding reduces quitting.

2.2 Supporting Metrics

Metric	Definition	Purpose
Activation rate	% of starters who finish onboarding and complete their first lesson	Confirms people are engaging, not just clicking through faster
Day-1 / Day-7 return rate	% of users who reopen the app 1 / 7 days later	Confirms retained interest, not just a rushed completion
Paid conversion rate	% of activated users who become paying subscribers	Ties the funnel directly to revenue
Time to first lesson	Time from onboarding start to first real lesson	Direct measure of friction removed

2.3 Guardrail Metrics (must not regress)

Guardrail	Risk if ignored
Lesson relevance / personalization quality	Fewer onboarding questions → generic content → quiet churn
Day-30 return rate	Shorter onboarding could attract low-intent users who finish easily but never stick
Refund / cancellation rate	Rushed onboarding → users don't understand what they signed up for
Onboarding-related support tickets	A skipped step (e.g. billing/permissions explanation) creates confusion
Notification opt-in rate	If deferred/removed, hurts our ability to re-engage users later

3. Data Platform Requirement

Answering “did the new onboarding work, and is it safe to ship” requires a pipeline that can: (1) collect every funnel and product event at the user level, tagged with test group/platform/timestamp; (2) validate and land it reliably; (3) model it into a warehouse that

supports both the primary metric and every guardrail; and (4) serve it to a dashboard and a statistical test. Assignments 1-4 build exactly this pipeline, end to end.

See [02_event_tracking_plan.md](#) for the full event schema and [data_generator.py](#) for the synthetic data used to develop and test the pipeline before real production data is available.

1. Design Principles

1. **One row per event** (long/narrow format) — flexible for any funnel shape, easy to append new event types without a schema migration.
2. **Every event carries the same envelope fields** so any event can be sliced by test group, platform, user and time without joins.
3. **Event-specific detail lives in a properties JSON blob** rather than dozens of mostly-null columns — kept narrow at the raw layer, flattened out later in staging/warehouse.
4. Event names are snake_case, past-tense, object_verb (e.g. onboarding_step_completed) for consistency and easy grouping.

2. Envelope (present on every event)

Field	Type	Description
event_id	UUID (string)	Unique id per event row
user_id	string	Anonymous/user identifier, stable per user
event_name	string	One of the catalog names in Section 4
event_timestamp	ISO-8601 datetime (UTC)	When the event occurred
test_group	enum A,B,C	Assigned onboarding variant
platform	enum iOS,Android	Device platform
session_id	string	Groups events within one app session
properties	JSON	Event-specific attributes (Section 4)

3. Identity & Randomization

- user_id is assigned at app_installed and is stable for the user's lifetime in the dataset.
- Test-group assignment happens once, at onboarding_started (first app open), via a uniform random draw across A/B/C (~33% each), stratified so iOS/Android are balanced within each group.

4. Event Catalog

4.1 Funnel & Onboarding Events

Event	Fires when	Key properties
app_installed	User installs the app	device_platform
onboarding_started	User begins onboarding	total_steps_in_flow
onboarding_step_viewed	User sees a step	step_number, step_name, step_type (must_have/can_wait/nice_to_have)
onboarding_step_completed	User completes a step	step_number, step_name, time_on_step_seconds
onboarding_step_abandoned	User exits mid-step	step_number, step_name, exit_reason
onboarding_completed	User finishes the whole flow	total_time_seconds, steps_completed
account_created	User creates an account/profile	signup_method

4.2 Product Engagement Events

Event	Fires when	Key properties
first_lesson_started	User begins first lesson (activation trigger)	lesson_id
first_lesson_completed	User finishes first lesson (activation success)	lesson_id, score
app_opened_day1 / app_opened_day7 / app_opened_day30	Return visit at that horizon	days_since_install
notification_permission_shown	Permission prompt shown	—
notification_permission_accepted / notification_permission_denied	User responds	—

4.3 Monetization Events

Event	Fires when	Key properties
trial_started	Free trial begins	trial_length_days
subscription_purchased	User converts to paying subscriber	plan, price_usd

subscription_cancelled	User cancels	days_since_purchase
refund_requested	User requests refund	days_since_purchase, reason

4.4 Quality & Guardrail Events

Event	Fires when	Key properties
app_crash / app_error	Technical issue	error_code, screen
support_ticket_created	Support ticket logged	is_onboarding_related (bool), category
lesson_feedback_submitted	In-app lesson rating	relevance_score (1-5)

5. Step Maps by Variant

Encodes the “Must-have / Can-wait / Nice-to-have” bucketing from the business document, used both by the generator and by the warehouse to explain *why* completion differs between groups.

Variant	Steps	Composition
A (control)	13	All must-have + can-wait + nice-to-have steps
B (medium)	7	All must-have + a subset of can-wait steps
C (short)	5	Must-have steps only

Full step list with bucket labels: `stepmap.json`.

6. Metric → Event Lineage

KPI	Computed from
Onboarding completion rate	onboarding_started vs onboarding_completed
Activation rate	onboarding_completed vs first_lesson_completed
Day-1 / Day-7 / Day-30 return rate	app_opened_day1/7/30 vs onboarding_started
Paid conversion rate	first_lesson_completed vs subscription_purchased
Time to first lesson	onboarding_started.timestamp → first_lesson_started.timestamp
Refund / cancellation rate	subscription_purchased vs refund_requested/subscription_cancelled
Notification opt-in rate	notification_permission_shown vs _accepted
Support ticket rate (onboarding)	support_ticket_created where is_onboarding_related

See [event_schema.json](#) for the machine-readable schema and [stepmap.json](#) for the per-variant step definitions, both consumed directly by [data_generator.py](#).